SICHERHEITS-AUDIT

NetMon Roots v1.1.14

Code Security Review — Juni 2026

Datum	28. Juni 2026
Analysierte Version	v1.1.14 — Arbeitsstand
Geaenderte / neue Dateien	80+
Neue Module	18
Kritische Findings	0
Bewertete Kandidaten	9
Konfidenz-Schwelle	>= 8 / 10
Methode	Statische Code-Analyse (Multi-Agenten)
Analysewerkzeug	Claude Sonnet 4.6 (Anthropic)
Projektverantwortlicher	Sven Froehlich

KEINE KRITISCHEN SCHWACHSTELLEN GEFUNDEN

1. Zusammenfassung

Die statische Sicherheitsanalyse des NetMon-Arbeitsstands v1.1.14 (80+ Dateien, davon 18 neue Module) durch ein automatisiertes Multi-Agenten-System auf Basis von Claude Sonnet 4.6 ergab keine ausnutzbaren Sicherheitsluecken mit echtem Angriffspfad. Alle neun initial identifizierten Kandidaten wurden in einer zweiten, unabhaengigen Filterrunde analysiert und als False Positives ausgeschlossen (Konfidenz < 8/10).

Erstmalig einbezogen wurden die neuen Module: Prozess-Guard, Repository-Guard, Roots-Baseline-Guard, Radio, Diagnostik, Support-Bundle, Links-Controller und weitere. Die zuvor in v1.1.11 behobenen fuenf Sicherheitsprobleme sind weiterhin geschlossen.

2. Methodik

Phase 1 — Reconnaissance

Lesen aller geaenderten und neuen Dateien; Identifikation sicherheitskritischer Muster: subprocess-Aufrufe, sudo-Verwendung, Dateipfad-Operationen mit Nutzereingabe, Deserialisierung (pickle, yaml), eval/exec, externe API-Aufrufe.

Phase 2 — Kandidaten-Analyse

Datenflussketten von Nutzereingaben bis zu sensitiven Operationen nachverfolgt. Geprueft: shell=True, Argument-Injection (list-form subprocess), Path-Traversal, Symlink-Bypasses und Privilege-Escalation-Pfade.

Phase 3 — False-Positive-Filter

Jeder Kandidat wurde von einem unabhaengigen Agenten re-analysiert. Ausschlusskriterien: Denial-of-Service, lokale Single-User-Desktops ohne Fremd-Input, reine Test-Dateien, Environment-Variablen als vertrauenswuerdig, list-form subprocess ohne shell=True, Clipboard-Inhalte ohne Auto-Execution.

Konfidenz-Schwelle: Nur Findings mit Score >= 8/10 waeren in den Abschlussbericht aufgenommen worden.

3. Analysierte Dateien (Auswahl)

Datei / Modul	Fokus	Ergebnis
`agent/runner.py`	Befehlsallowlist, subprocess	Kein Finding
`agent/tools.py`	Dateizugriff, Pfadvalidierung	Kein Finding
`netmon/features/firewall_controller.py`	UFW-Kommandos, Killswitch	Kein Finding
`netmon/features/packet_capture_controller.py`	BPF-Filter, tshark	Kein Finding
`netmon/features/vpn_controller.py`	WireGuard, nmcli, Profil-Validierung	Kein Finding
`netmon/features/hdd_controller.py`	Mountpoint-Validierung, sudo mount	Kein Finding

netmon/features/clamav_controller.py	Tempdir, freshclam	Kein Finding
netmon/features/security_monitoring_controller/	Integritaetspruefung, Baseline	Kein Finding
netmon/features/process_guard_controller.py	Prozessueberwachung, subprocess	Kein Finding
netmon/features/repository_guard_controller.py	APT-Quellen, Snippet-Generierung	Kein Finding
netmon/features/roots_baseline_guard_controller.py	sudo cat, fnmatch-Allowlist	Kein Finding
netmon/features/links_controller.py	Link-Verwaltung, URL-Handling	Kein Finding
netmon/services/ai_online_service.py	OpenAI-API, Dateianhang	Kein Finding
netmon/services/dns_control_service.py	DNS-Token-Validierung, nmcli	Kein Finding
netmon/services/process_guard_service.py	Prozess-Scan, ps-Aufrufe	Kein Finding
netmon/services/repository_guard_service.py	APT-Quelldateien, dpkg, Flatpak	Kein Finding
netmon/core/lock.py	PID-Lock, EPERM-Behandlung	Kein Finding
netmon/core/rkhunter_manager.py	sudo rkhunter, WEB_CMD	Kein Finding
netmon/core/diagnostics.py	System-Diagnose, Dateisammlung	Kein Finding
netmon/core/support_bundle.py	Log-Archivierung, Pfad-Handling	Kein Finding
netmon/services/startup_guard_service.py	Baseline, Hash-Validierung	Kein Finding
netmon/services/path_guard_service.py	PATH-Eintrags-Analyse	Kein Finding
netmon/features/dns_controller.py	DNS-Guard, DoT-Konfiguration	Kein Finding

4. Bewertete Kandidaten (alle ausgeschlossen)

Alle neun initial identifizierten Muster wurden re-analysiert und als False Positives ausgeschlossen (Konfidenz-Score < 8/10).

K1 — Git-Subcommand-Passthrough

Konfidenz: 4/10

`Datei: agent/runner.py`

git ist in der Befehlsallowlist. Ohne separate Subcommand-Allowlist koennte ein Angreifer git diff --no-index oder git -c core.sshCommand einschleusen.

> Filterentscheid: AUSGESCHLOSSEN: runner.py enthaelt einen expliziten Subcommand-Check (ALLOWED_GIT_SUBCOMMANDS). Einziger Aufrufer ist agent/cli.py via sys.argv des Entwicklers — vertrauenswuerdige Eingabe. Kein LLM-Output und kein Netzwerk-Input erreicht die Funktion.

K2 — WireGuard-Endpoint in UFW-Killswitch

Konfidenz: 3/10

`Datei: netmon/features/firewall_controller.py`

parse_vpn_endpoint() gibt den Hostname ohne IP-Validierung zurueck. Endpoint-Wert 'any' wuerde in ufw allow out to any port 51820 resultieren und den Killswitch untergraben.

> Filterentscheid: AUSGESCHLOSSEN: WireGuard akzeptiert 'any' nicht als validen Endpoint. /etc/wireguard/*.conf ist root-owned; Einschleusen benoetigt bereits Root-Rechte. Kein fernausnutzbarer Pfad.

K3 — UFW-Quelle 'any' im Firewall-Regelformular

Konfidenz: 2/10

`Datei: netmon/features/firewall_controller.py`

_is_valid_source_token() akzeptiert 'any' explizit, was in einer vollstaendig offenen UFW-Regel resultieren kann.

> Filterentscheid: AUSGESCHLOSSEN: Gewolltes Verhalten einer Firewall-Verwaltungs-GUI. UI zeigt Warnung, Live-Vorschau und askyesno-Dialog. Single-User-Desktop; Operator und Admin sind dieselbe Person.

K4 — Vorhersagbarer Temp-Pfad fuer ClamAV-Datenbank

Konfidenz: 3/10

`Datei: netmon/features/clamav_controller.py`

Pfad /tmp/netmon-clamav-db-{username} ist vorhersagbar. os.makedirs(exist_ok=True) folgt Symlinks.

> Filterentscheid: AUSGESCHLOSSEN: freshclam schreibt ausschliesslich kryptographisch signierte ClamAV-Signaturdateien (.cvd/.cld). Angreifer kontrolliert wohin, nicht was. Keine Code-Ausfuehrung moeglich. DoS ausgeschlossen.

K5 — Dateipfad in image_to_data_url()

Konfidenz: 2/10

`Datei: netmon/services/ai_online_service.py`

image_to_data_url(path) koennte theoretisch beliebige Dateien base64-kodiert an die OpenAI-API senden.

> Filterentscheid: AUSGESCHLOSSEN: Dateiendungs-Allowlist blockiert Nicht-Bild-Dateien. Dateien kommen ausschliesslich aus dem OS-Dateidialog. Zwei Bestaertigungsdialoge vor API-Uebertragung vorhanden.

K6 — WireGuard-Profil-Import via nmcli

Konfidenz: 3/10

`Datei: netmon/features/vpn_controller.py`

import_wireguard_paths_to_networkmanager() uebergibt den OS-Dateidialog-Pfad an 'sudo nmcli connection import type wireguard file '. PostUp/PreDown-Hooks koennten Schadcode ausfuehren.

> Filterentscheid: AUSGESCHLOSSEN: NetworkManager verwendet den Kernel-WireGuard-Treiber; PostUp/PreDown sind wg-quick-spezifisch und werden von nmcli nicht ausgefuehrt. Pfad kommt aus dem nativen OS-Dateidialog.

K7 — WireGuard-Profil-Import via wg-quick

Konfidenz: 3/10

`Datei: netmon/features/vpn_controller.py`

import_wg_profiles_from_folder() kopiert per 'sudo install -m 600' eine nutzerwaehlte Datei nach /etc/wireguard/. PostUp-Hooks wuerden bei naechstem Verbindungsaufbau als Root laufen.

> Filterentscheid: AUSGESCHLOSSEN: Quelldatei kommt aus OS-Dateidialog. Kein automatisches wg-quick up folgt; Nutzer muss VPN-Slot manuell aktivieren. PostUp-Hooks als Root sind das dokumentierte WireGuard-Design. Nutzer == Operator.

K8 — Shell-Snippet-Generierung im Repository-Guard

Konfidenz: 2/10

`Datei: netmon/features/repository_guard_controller.py`

get_repository_guard_quick_action_snippet() bettet APT-Quelldatei-Werte (URI, Host, Pfad) per f-String in Shell-Befehle ein und kopiert das Ergebnis in die Zwischenablage.

> Filterentscheid: AUSGESCHLOSSEN: Snippet wird von der Anwendung nicht ausgefuehrt — nur in die Zwischenablage kopiert. Einschleusen einer praeparierten APT-Quelldatei erfordert bereits Root-Rechte. Kein Auto-Exec-Pfad.

K9 — Symlink-Bypass im Roots-Baseline-Guard

Konfidenz: 3/10

`Datei: netmon/features/roots_baseline_guard_controller.py`

_is_allowed_guard_path() verwendet os.path.abspath statt os.path.realpath, sodass ein Symlink unter erlaubtem Pfadmuster den fnmatch-Check bestehen wuerde. _capture_file_state_with_root() liest Symlink-Ziel per sudo cat.

Filterentscheid: AUSGESCHLOSSEN: /etc/shadow liegt bereits explizit in ROOTS_GUARD_DEFAULT_AREA_PATHS — das hypothetische Ziel ist ohnehin im Scope. Pfade aus hartkodierter Konfiguration ohne Nutzereingabe. Single-User-Desktop: Nutzer == Admin == Operator.

5. Im Codestand behobene Sicherheitsprobleme

Die folgenden Sicherheitsprobleme wurden im Commit-Bereich e334324a ... 920cfc9b behoben und sind im analysierten Codestand nicht mehr vorhanden.

Mountpoint Path Traversal — hdd_controller.py Commit: 0431faf8

Ohne Validierung konnte ein Angreifer mit Schreibzugriff auf die HDD-Slot-Konfiguration einen beliebigen Mountpoint (z. B. /etc) setzen, der per 'sudo mount' mit einem praeparierten Filesystem ueberschrieben werden konnte (Privilege Escalation). Fix: os.path.realpath() zur Kanonisierung; Pruefung gegen _SAFE_MOUNT_BASES (/mnt/, /media/, /run/media/).

Command Injection via CWD — linux_help_controller.py Commit: 363d64f5

os.getcwd() war im CSV-Suchpfad-Fallback enthalten. Eine manipulierte CSV-Datei aus einem world-writable Verzeichnis haette Command Injection ermoeglichen koennen. Fix: CWD-Fallback entfernt.

Lock-Bypass via EPERM — lock.py Commit: 363d64f5

EPERM von os.kill() wurde faelschlich als veralteten Lock behandelt, was eine zweite Instanz als anderer User ermoeglichen konnte. Fix: EPERM korrekt als 'Prozess laeuft, anderer User' behandelt.

DNS Token Zero-Bug — dns_control_service.py Commit: 363d64f5

Falsy-Zero-Falle: token.count('.') war truthy-getestet, wodurch ein Token mit null Punkten als gueltig durchging. Fix: token.count('.') > 0.

Startup Guard Baseline-Blindspot — startup_guard_service.py Commit: 363d64f5

Neue Dateien ohne Baseline-Eintrag wurden nach einem Baseline-Reset unsichtbar, wenn sie in der Trustlist standen. Fix: neue Dateien immer als 'new' gemeldet, unabhaengig vom Trustlist-Status.

6. Hardening-Empfehlungen (keine kritischen Bugs)

Die folgenden Punkte sind keine Sicherheitsluecken, sondern Defense-in-Depth-Massnahmen fuer kuenftige Entwicklungsrunden.

1. Git-Subcommand-Allowlist in agent/runner.py

Sobald LLM-Output in run_command() fliesst, Subcommand-Allowlist ergaenzen (status, diff, log, show). Derzeit latentes Architektur-Risiko.

2. IP-Validierung fuer WireGuard-Endpoints

parse_vpn_endpoint() sollte den Host-Teil mit ipaddress.ip_address() validieren bevor er in UFW-Kommandos eingebettet wird.

3. Pfadpruefung in image_to_data_url()

Optional: Dateipfad zusaetzlich auf das Benutzer-Homeverzeichnis einschraenken. Aktuell kein ausnutzbarer Angriffspfad vorhanden.

4. ClamAV Tempdir gegen Symlink-Angriffe haerten

os.makedirs() koennte durch os.lstat() + Fehler bei Symlink gehaertet werden.

5. Shell-Quoting in Repository-Guard-Snippets

URI- und Pfadwerte in get_repository_guard_quick_action_snippet() sollten mit shlex.quote() maskiert werden. Verhindert Snippet-Inkonsistenzen bei Sonderzeichen in APT-Quelldateien.

6. os.path.realpath() in Roots-Baseline-Guard

_normalize_guard_path_pattern() sollte os.path.abspath durch os.path.realpath() ersetzen, um Symlinks vor dem fnmatch-Check aufzuloesen.

7. Geprufte Sicherheitskategorien

Kategorie	Ergebnis
Command Injection (subprocess, shell=True)	Kein Finding
Path Traversal (Dateioperationen mit Nutzereingabe)	Kein Finding
Privilege Escalation (sudo, Argument-Injektion)	Kein Finding
Unsichere Deserialisierung (pickle, yaml.load)	Kein Finding
Eval/Exec mit Nutzereingabe	Kein Finding
Hardcoded Credentials / API-Keys	Kein Finding
Authentication Bypass / Privilege Escalation	Kein Finding
Sensitive Data Exposure (PII in Logs)	Kein Finding
Externe API-Aufrufe mit unkontrollierten Daten	Kein Finding
Kryptographische Schwachstellen	Kein Finding
Symlink-Angriffe auf privilegierte Dateioperationen	Kein Finding
Clipboard-Injection / Snippet-Generierung ohne Auto-Exec	Kein Finding

Impressum und Haftungsausschluss

Auditor: Claude Sonnet 4.6 (Anthropic) — Automatisiertes Multi-Agenten-Sicherheits-Review

Projektverantwortlicher: Sven Froehlich

Datum: 28. Juni 2026

Analysierter Codestand: e334324a ... 920cfc9b + Arbeitsstand (unstaged), gesamt 80+ Dateien

Methode: Statische Code-Analyse — kein Laufzeit-Penetrationstest

Dieses Audit ersetzt keinen professionellen Penetrationstest durch eine zertifizierte Sicherheitsfirma. Die Analyse beschraenkt sich auf den angegebenen Codestand. Nicht erfasst werden: Laufzeitverhalten, abhaengige Systembibliotheken, Netzwerkkonfiguration und soziale Angriffsvektoren.